Vending Machine Using State Pattern Pattern

- Definition
- Real Life Example: Traffic Signal
- State Design Pattern UML:
  - Lets Code this UML for Traffic Signal:
- Real Life Example: Vending Machine
  - Understanding the working of a Vending Machine
  - Different States and Operations
    - Example: Vending Machine
- Vending Machine representation using State Pattern
- Implementation(e.g., Vending Machine)

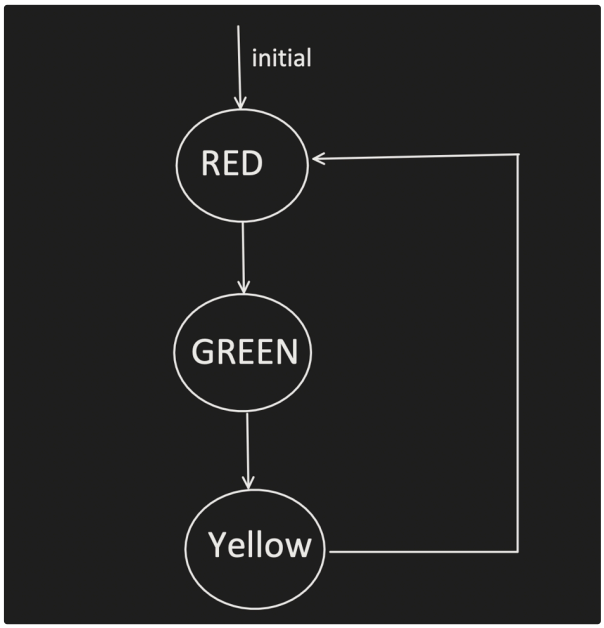
Resources

- Video → 41. All Behavioral Design Patterns | Strategy, Observer, State, Template, Command, Visitor, Memento
- Video → 16. Design Vending Machine (Hindi) | LLD of Vending Machine | State Design Pattern | LLD question

Definition

> **The State Pattern allows an object to change its behavior dynamically at runtime whenever there is a change in its internal state.**

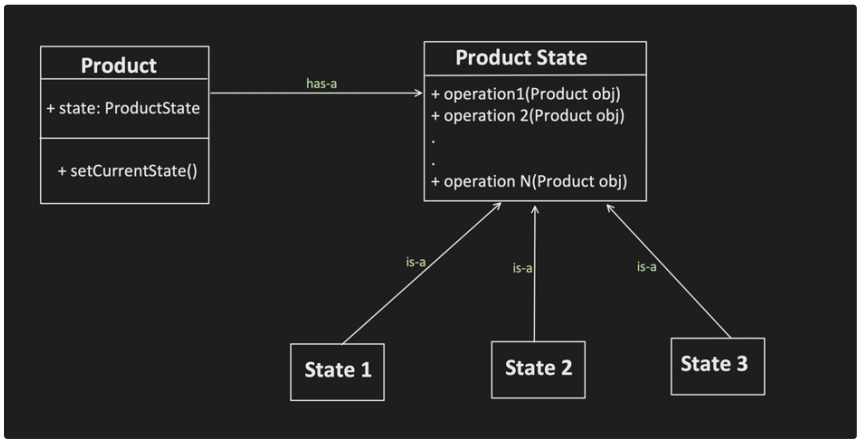
Real Life Example: Traffic Signal



State	Operations
Red State (Stop)	action() -> Make Signal Red
Green State (Go)	action() ->Make Signal Green
Yellow State (Slow Down)	action() ->Make Signal Yellow

This type of problems, where Object change the state after performing certain operation can be solved through : State Design Pattern

State Design Pattern UML:



Lets Code this UML for Traffic Signal:

```java
public interface TrafficLightState {
    void action(TrafficLight signal);
}

```

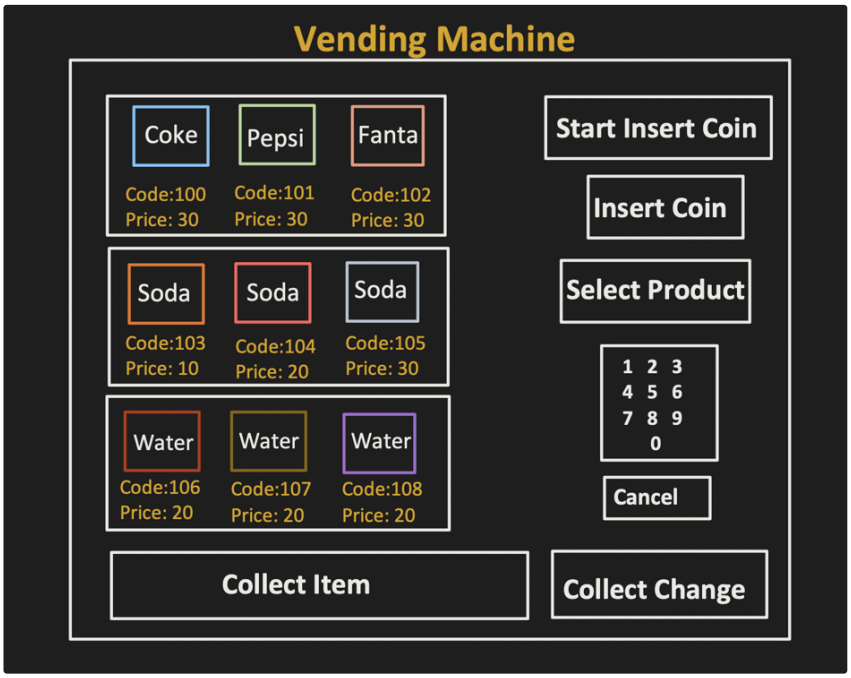
```java
public class RedState implements TrafficLightState {

   @Override
    public void action(TrafficLight signal) {
        //STOP behavior
        signal.setState(new GreenState()); // next state
    }
}


public class GreenState implements TrafficLightState {

  @Override
    public void action(TrafficLight signal) {
        //GO behavior
        signal.setState(new YellowState()); // next state
    }
```

```java
}


public class YellowState implements TrafficLightState {

  @Override
    public void action(TrafficLight signal) {
        //Slow Down behavior
        signal.setState(new RedState()); // next state
    }
}


```

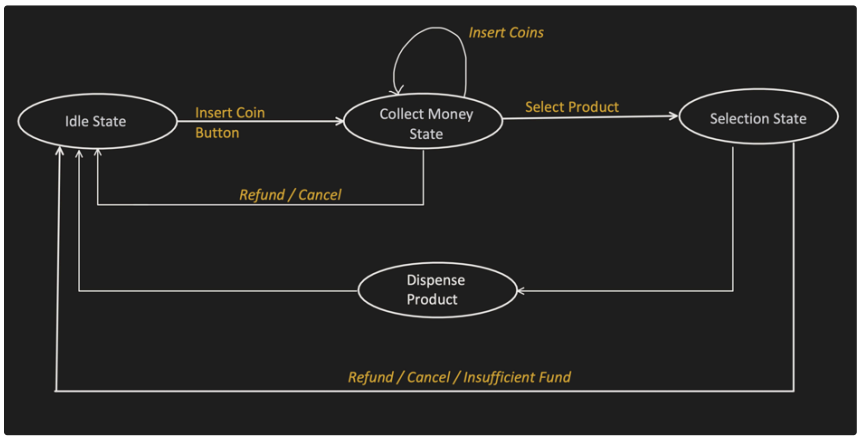
```java
public class TrafficLight {
    private TrafficLightState state;

    public TrafficLight() {
        this.state = new RedState(); // initial state
    }

    public void setState(TrafficLightState state) {
        this.state = state;
    }

    public void change() {
        state.action(this);
    }
}

```

```java
public class Main {
    public static void main(String[] args) {
        TrafficLight trafficLight = new TrafficLight (); //initial signal state is RED

        trafficLight.change(); // RED → GREEN
        trafficLight.change(); // GREEN → YELLOW
        trafficLight.change(); // YELLOW → RED
    }
}

```

Real Life Example: Vending Machine



Understanding the working of a Vending Machine



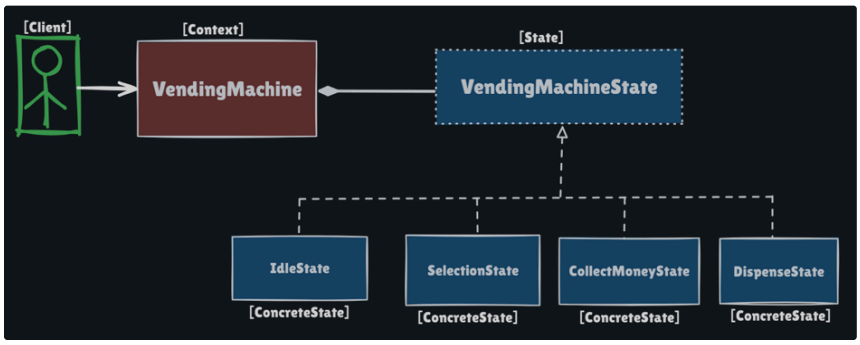
Different States and Operations

Example: Vending Machine

State	Operations
IdleState	• Press Insert Coin button
HasMoney State	• Insert Coin • Select Product • Cancel / Refund
Selection State	• Choose Product • Return change

	• Cancel/Refund full amount
Dispense Product State	• Dispense Product

Vending Machine representation using State Pattern



1. **State Interface(e.g., `VendingMachineState`):** Declares common functions that all states must implement.
2. **Concrete States(e.g., `IdleState`, `SelectionState`, `hasMoney`, `DispenseState`):** Each class implements the state interface behaviors(operations) differently depending on the current state of the vending machine, and an exception is thrown for operations that do not apply to the current state.
3. **Context Class** (e.g., `VendingMachine`): Maintains a reference to the current state. Holds all possible states as objects. Delegates call to the current state object.
4. **Client(`VendingMachineAppDemo`):** Interacts with Context Class (`VendingMachine`) and expects appropriate behavior as per changes in the state of the object.

Implementation(e.g., Vending Machine)

```java
public abstract class State {

    public void clickOnInsertCoinButton(VendingMachine machine) throws Exception {
         // by default nothing happens
    }

    public void clickOnStartProductSelectionButton(VendingMachine machine) throws Exception {
        // by default nothing happens
    }

    public void insertCoin(VendingMachine machine, Coin coin) throws Exception {
        // by default nothing happens
    }

    public void chooseProduct(VendingMachine machine, int codeNumber) throws Exception {
        // by default nothing happens
    }

    public int getChange(int returnChangeMoney) throws Exception {
        // by default nothing happens
        return 0;
    }

    public Item dispenseProduct(VendingMachine machine, int codeNumber) throws Exception {
        // by default nothing happens
```

```java
        return null;
    }

    public List<Coin> refundFullMoney(VendingMachine machine) throws Exception {
        // by default nothing happens
        return null;
    }

    public void updateInventory(VendingMachine machine, Item item, int codeNumber) throws Exception {
        // by default nothing happens
    }
}
```

```java
public class IdleState extends State {

    public IdleState(){
        System.out.println("Currently Vending machine is in IdleState");
    }

    public IdleState(VendingMachine machine){
        System.out.println("Currently Vending machine is in IdleState");
        machine.setCoinList(new ArrayList<>());
    }

    @Override
    public void clickOnInsertCoinButton(VendingMachine machine) throws Exception{
        machine.setVendingMachineState(new HasMoneyState());
    }

    @Override
    public void updateInventory(VendingMachine machine, Item item, int codeNumber) throws Exception {
        machine.getInventory().addItem(item, codeNumber);
    }
}
```

```java
public class HasMoneyState extends State {

    public HasMoneyState(){
        System.out.println("Currently Vending machine is in HasMoneyState");
    }

    @Override
    public void clickOnStartProductSelectionButton(VendingMachine machine) throws Exception {
        machine.setVendingMachineState(new SelectionState());
    }

    @Override
    public void insertCoin(VendingMachine machine, Coin coin) throws Exception {
        System.out.println("Accepted the coin");
        machine.getCoinList().add(coin);
    }

    @Override
    public List<Coin> refundFullMoney(VendingMachine machine) throws Exception {
        System.out.println("Returned the full amount back in the Coin Dispense Tray");
        machine.setVendingMachineState(new IdleState(machine));
        return machine.getCoinList();
```

```
    }
}
```

```java
public class SelectionState extends State {

    public SelectionState(){
        System.out.println("Currently Vending machine is in SelectionState");
    }

    @Override
    public void chooseProduct(VendingMachine machine, int codeNumber) throws Exception{

        //1. get item of this codeNumber
        Item item = machine.getInventory().getItem(codeNumber);

        //2. total amount paid by User
        int paidByUser = 0;
        for(Coin coin : machine.getCoinList()){
            paidByUser = paidByUser + coin.value;
        }

        //3. compare product price and amount paid by user
        if(paidByUser < item.getPrice()) {
            System.out.println("Insufficient Amount, Product you selected is for price: " + item.getPrice() + " and you paid: " + paidByUser);
            refundFullMoney(machine);
            throw new Exception("insufficient amount");
        }
        else if(paidByUser >= item.getPrice()) {

            if(paidByUser > item.getPrice()) {
                getChange(paidByUser-item.getPrice());
            }
            machine.setVendingMachineState(new DispenseState(machine, codeNumber));
        }
    }

    @Override
    public int getChange(int returnExtraMoney) throws Exception{
        //actual logic should be to return COINs in the dispense tray, but for simplicity i am just returning the amount to be refunded
        System.out.println("Returned the change in the Coin Dispense Tray: " + returnExtraMoney);
        return returnExtraMoney;
    }

    @Override
    public List<Coin> refundFullMoney(VendingMachine machine) throws Exception{
        System.out.println("Returned the full amount back in the Coin Dispense Tray");
        machine.setVendingMachineState(new IdleState(machine));
        return machine.getCoinList();
    }
}
```

```java
public class DispenseState extends State {

    DispenseState(VendingMachine machine, int codeNumber) throws Exception{
        System.out.println("Currently Vending machine is in DispenseState");
        dispenseProduct(machine, codeNumber);
    }

    @Override
```

```java
    public Item dispenseProduct(VendingMachine machine, int codeNumber) throws Exception{
        System.out.println("Product has been dispensed");
        Item item = machine.getInventory().getItem(codeNumber);
        machine.getInventory().updateSoldOutItem(codeNumber);
        machine.setVendingMachineState(new IdleState(machine));
        return item;
    }
}


```

```java
public class VendingMachine {

    private State vendingMachineState;
    private Inventory inventory;
    private List<Coin> coinList;

    public VendingMachine(){
        vendingMachineState = new IdleState();
        inventory = new Inventory(10);
        coinList = new ArrayList<>();
    }

    public State getVendingMachineState() {
        return vendingMachineState;
    }

    public void setVendingMachineState(State vendingMachineState) {
        this.vendingMachineState = vendingMachineState;
    }

    public Inventory getInventory() {
        return inventory;
    }

    public void setInventory(Inventory inventory) {
        this.inventory = inventory;
    }

    public List<Coin> getCoinList() {
        return coinList;
    }

    public void setCoinList(List<Coin> coinList) {
        this.coinList = coinList;
    }
}
```

```java
public enum Coin {

    PENNY(1),
    NICKEL(5),
    DIME(10),
    QUARTER(25);

    public int value;

    Coin(int value) {
        this.value = value;
    }
}

```

```java
public class Inventory {

```

```java
    ItemShelf[] inventory = null;

    Inventory(int itemCount) {
        inventory = new ItemShelf[itemCount];
        initialEmptyInventory();
    }

    public ItemShelf[] getInventory() {
        return inventory;
    }

    public void setInventory(ItemShelf[] inventory) {
        this.inventory = inventory;
    }

    public void initialEmptyInventory() {
        int startCode = 101;
        for (int i = 0; i < inventory.length; i++) {
            ItemShelf space = new ItemShelf();
            space.setCode(startCode);
            space.setSoldOut(true);
            inventory[i]= space;
            startCode++;
        }
    }

    public void addItem(Item item, int codeNumber) throws Exception {

        for (ItemShelf itemShelf : inventory) {
            if (itemShelf.code == codeNumber) {
                if (itemShelf.isSoldOut()) {
                    itemShelf.item = item;
                    itemShelf.setSoldOut(false);
                } else {
                    throw new Exception("already item is present, you can not add item here");
                }
            }
        }
    }

    public Item getItem(int codeNumber) throws Exception {

        for (ItemShelf itemShelf : inventory) {
            if (itemShelf.code == codeNumber) {
                if (itemShelf.isSoldOut()) {
                    throw new Exception("item already sold out");
                } else {

                    return itemShelf.item;
                }
            }
        }
        throw new Exception("Invalid Code");
    }

    public void updateSoldOutItem(int codeNumber){
        for (ItemShelf itemShelf : inventory) {
            if (itemShelf.code == codeNumber) {
                itemShelf.setSoldOut(true);
            }
        }
    }
}
```

```java
public class ItemShelf {

    int code;
    Item item;
```

```java
    boolean soldOut;

    public int getCode() {
        return code;
    }

    public void setCode(int code) {
        this.code = code;
    }

    public Item getItem() {
        return item;
    }

    public void setItem(Item item) {
        this.item = item;
    }

    public boolean isSoldOut() {
        return soldOut;
    }

    public void setSoldOut(boolean soldOut) {
        this.soldOut = soldOut;
    }
}
```

```java
public class Item {
    ItemType type;
    int price;

    public ItemType getType() {
        return type;
    }

    public void setType(ItemType type) {
        this.type = type;
    }

    public int getPrice() {
        return price;
    }

    public void setPrice(int price) {
        this.price = price;
    }
}


public enum ItemType {

    COKE,
    PEPSI,
    JUICE,
    SODA;
}

```

```java
public class VendingMachineAppDemo {

    public static void main(String args[]){

        VendingMachine vendingMachine = new VendingMachine();
        try {

            System.out.println("|");
            System.out.println("filling up the inventory");
            System.out.println("|");
```

```java

            fillUpInventory(vendingMachine);
            displayInventory(vendingMachine);

            System.out.println("|");
            System.out.println("clicking on InsertCoinButton");
            System.out.println("|");

            State vendingState = vendingMachine.getVendingMachineState();
            vendingState.clickOnInsertCoinButton(vendingMachine);

            vendingState = vendingMachine.getVendingMachineState();
            vendingState.insertCoin(vendingMachine, Coin.NICKEL);
            vendingState.insertCoin(vendingMachine, Coin.QUARTER);
            // vendingState.insertCoin(vendingMachine, Coin.NICKEL);

            System.out.println("|");
            System.out.println("clicking on ProductSelectionButton");
            System.out.println("|");

vendingState.clickOnStartProductSelectionButton(vendingMachine);

            vendingState = vendingMachine.getVendingMachineState();
            vendingState.chooseProduct(vendingMachine, 102);

            displayInventory(vendingMachine);

        }
        catch (Exception e){
            displayInventory(vendingMachine);
        }


    }

    private static void fillUpInventory(VendingMachine vendingMachine) {
        ItemShelf[] slots = vendingMachine.getInventory().getInventory();
        for (int i = 0; i < slots.length; i++) {
            Item newItem = new Item();
            if(i >=0 && i<3) {
                newItem.setType(ItemType.COKE);
                newItem.setPrice(12);
            }else if(i >=3 && i<5){
                newItem.setType(ItemType.PEPSI);
                newItem.setPrice(9);
            }else if(i >=5 && i<7){
                newItem.setType(ItemType.JUICE);
                newItem.setPrice(13);
            }else if(i >=7 && i<10){
                newItem.setType(ItemType.SODA);
                newItem.setPrice(7);
            }
            slots[i].setItem(newItem);
            slots[i].setSoldOut(false);
        }
    }

    private static void displayInventory(VendingMachine vendingMachine){

        ItemShelf[] slots = vendingMachine.getInventory().getInventory();
        for (int i = 0; i < slots.length; i++) {

            System.out.println("CodeNumber: " + slots[i].getCode() +
                    " Item: " + slots[i].getItem().getType().name() +
                    " Price: " + slots[i].getItem().getPrice() +
                    " isAvailable: " + !slots[i].isSoldOut());
```

```
        }
    }

}
```